Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 14 (MidTerm):
Sviluppo di un Sistema Bancario TCP Multithread

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Introduzione al Problema **2**

2 Stato dell'Arte **2**

3 Metodologia usata **2**
3.1 Progettazione del Database 2
3.2 Protezione Logica e Limitazione Server 2

4 Risultati Sperimentali **3**
4.1 Analisi di Rete (Wireshark) 4

5 Conclusione e Sviluppi Futuri **4**

1 Introduzione al Problema

Il MidTerm (Hands-On 14) riunisce ed eleva le competenze acquisite operando finora con i Socket Berckley e con le astrazioni della libreria `Pthreads`. La richiesta principale è l'implementazione in linguaggio C di un **Sistema Bancario** minimale, in grado di gestire un correntista che invia comandi CRUD asincroni da molteplici istanti d'accesso (Terminali differenti, App, E-banking sportello). Il sistema si fonda su un'architettura **Client-Server TCP** e prevede limitatori rigorosi per la concorrenza (10 Threads), più protezioni di memoria contro la *Race Condition* che andrebbe a distruggere la congruenza dell'estratto conto.

2 Stato dell'Arte

Architetture come quella proposta si pongono alla base dei moderni layer di accesso Cloud.

- **TCP Connection-Oriented**: A differenza dell'UDP (Connectionless), la TCP Handshake assicura che il traffico bancario sia controllato, privo di perdita pacchetti (*Acknowledgment*) ed in corretto ordine seriale.
- **Gestione Demultiplexer-Worker**: Il design pattern adottato è quello *One-Thread-Per-Connection*. Il `main()` accetta pacchetti SYN, istanzia la socket e ne *esporta* il File Descriptor allocando un Worker Thread figlio staccato (Detached).
- **Critical Section**: Le operazioni di bilanciamento conto (Lettura-Modifica-Scrittura) non sono atomiche ai cicli di clock. Il DB (simulato) sarà interamente protetto tramite il costrutto `pthread_mutex_t`.

3 Metodologia usata

3.1 Progettazione del Database

La struttura `Transaction` immagazzina l'ID, la causale e l'importo. Un array `db[MAX_RECORDS]` simula la tabella centrale di archiviazione in backend. La discriminante principale è lo stato logico di esistenza.

Struttura Transazione

```c
typedef struct {
    int id;
    float amount;
    char reason[256];
    int active;
} Transaction;
```

3.2 Protezione Logica e Limitazione Server

Per obbedire al vincolo "Al massimo 10 thread", abbiamo implementato un semaforo limitante pre-accept:

Limitatore di traffico Server

```
// Inizializzato a MAX_NUM (10)
sem_wait(&thread_limiter);
client_sock = accept(server_sock, ...);
// Viene passata la richiesta a pthread_create
```

Ogniqualvolta una richiesta accede al DB, acquisisce un `pthread_mutex_lock(&db_lock)`. Se è un inserzione (`ADD`), scansiona i duplicati di ID per prevenire collisioni logiche. Lo stream testuale inviato dai socket viene parsate comodamente tramite `sscanf`.

4 Risultati Sperimentali

Innescando il demone server ed aprendo il client tramite l'eseguibile `./client`, la suite risponde correttamente con l'estratto CRUD implementato:

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho14$
   ./client
--- Connected to Bank Server ---
Available commands:
  ADD <id> <amount> <reason>
  UPDATE <id> <amount> <reason>
  DELETE <id>
  LIST
  QUIT
BANK> ADD 1 1500.00 Stipendio Mese Marzo
OK: ADDED 1
BANK> ADD 2 55.40 Rimborso Pizza
OK: ADDED 2
BANK> LIST
--- TRANSACTION LIST ---
[1] 1500.00 - Stipendio Mese Marzo
[2] 55.40 - Rimborso Pizza
------------------------
BANK> UPDATE 2 -20.00 Storno Parziale
OK: UPDATED 2
BANK> DELETE 1
OK: DELETED 1
BANK> LIST
--- TRANSACTION LIST ---
[2] -20.00 - Storno Parziale
------------------------
BANK> QUIT
```

L'architettura garantisce una stabilità duratura. La chiusura dell'innesco 'client' deallocherà silenziosamente il file descriptor riconsegnando un token libero al `thread_limiter`.

4.1 Analisi di Rete (Wireshark)

I test condotti sull'interfaccia di Loopback confermano a livello ISO/OSI il corretto funzionamento dell'inizializzazione Client/Server.

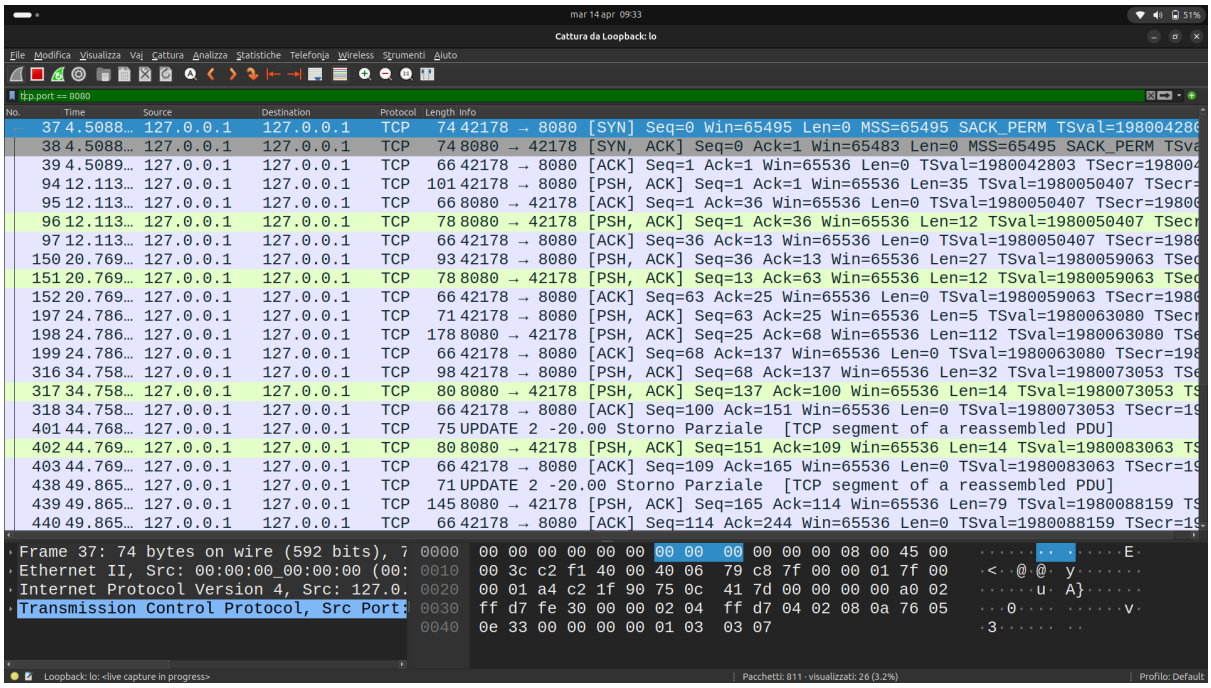

Figure 1: Cattura Wireshark: TCP 3-Way Handshake, Invio Dati (PSH) e Teardown

Come richiesto dal tracciato teorico, si osservano chiaramente le tre iterazioni fondamentali gestite dallo strato TCP: 1. L'avvio con l'Handshake a tre vie (`[SYN]`, `[SYN, ACK]` da parte del server, e l'ultimo `[ACK]`) che instaura il tunnel virtuale sicuro. 2. L'invio bidirezionale del payload mediante pacchetti identificati dal flag `[PSH, ACK]`, dove si cela l'operazione `ADD` inviata in chiaro. 3. Lo smantellamento concordato alla pressione del `QUIT` pilotato dal flag `[FIN, ACK]`.

5 Conclusione e Sviluppi Futuri

A un livello di Proof-Of-Concept, la gestione è completamente coerente: non permette transazioni duplici, non permette cancellazioni inesatte e parallelizza correttamente client multipli.
Sviluppi futuri ideali:

- **DB Storage Persistente**: Abbandono dell'array per delegare salvataggi espliciti su file `.txt` o database relazionali veri e propri (PostgreSQL, SQLite), così il riavvio del server non annienta le operazioni precedenti.
- **Read-Write Locks**: Utilizzo di `pthread_rwlock_t` al posto del grezzo mutex per far sì che molteplici chiamate `LIST` possano leggere senza bloccarsi vicendevolmente nel caso di estremo traffico in sola visualizzazione.